

Advanced iOS Development Assessment 3 - Project 2: Platform-Integrated Solution

Jaiden Gardner - 24833523

Section 1 - Problem Statement

What is the problem?

The current problem is that in small business environments dedicated security teams are often not realistic for budgets, meaning the responsibility of security falls on the security point of contact. Phishing and suspicious content that arrives over email or over the internet are amongst the most prevalent attacks in modern day and without a dedicated team, if an employee encountered suspicious activity responsibility falls on them to record these incidents. This creates friction as without a structured security tool, reporting these incidents becomes dependent upon this employee's dedication and memory, leading to informal notes that have no deadline or severity attached, and get easily lost track of. This friction coalesces into a major industry problem that contributes to attacks going unnoticed.

Who is the primary stakeholder?

The primary stakeholder is a security point of contact within a small organisation. This is a named role for businesses without a formal security team that are responsible for managing risks and security.

What currently happens when this problem is not solved?

Without a solution currently the human cost is time, risk, missed information and friction across the small organisation. Time is wasted as without dedicated records duplicate instances of suspicious activity can occur leading to re-investigating the same incident. Risk arises as real threats can go unresolved or unnoticed when they should have been escalated, this in turn produces missed information as there is no record of what was decided or what happened for similar cases. All together this creates friction, leading to the process of noticing, flagging and resolving threats becoming slow, ignored and unsafe.

Real-world source

Paloalto Networks. (2026). *Global Incident Report 2026*. Unit 42.

https://www.paloaltonetworks.com/resources/research/unit-42-incident-response-report?utm_source=google-panw_inhouse-japac-unit42-unrc-unpt&utm_medium=paid_search&utm_campaign=google-unit42-unit42_port-japac-anz-discovery-en-nonbrand-phr-t2-ThreatIntel&utm_content=701Ki000000h65BIAQ&utm_term=cyber%20crime%20threats&cg_plac=&cg_net=g&gclidsrc=aw.ds&gad_source=1&gad_campaignid=24092043248&gbraid=0AAAAADHVeKnrX9KKxbB0V5Vy8Ug9RmMu&gclid=Cj0KCQjwz4LWBhCMARIsAFEG5MpDXrXB2R-1F5_T9FMyVoqhiapkDIjfvFcFm_VMasNe_8zVI7F_VKkaAmF8EALw_wcB

Section 2 - Design Justification

iOS App

An iOS application acts as an ideal solution for the identified problem due to the ease of use it provides through its capabilities and unique focus compared to existing tools. Offline access through local databases allow the app to function independently and more securely, making it preferable to a website. While extension capabilities such as the widget extension allow for easy monitoring of content. The decision for a mobile app over a desktop application is not required as the app could function on either, but the portable nature of phones makes a mobile application preferable as the app can be opened anywhere easily, allowing for quick and efficient responses to any incidents. Compared to the alternative of using basic note taking tools, the specialisation of a dedicated app allows for more precise recording of incidents with the relevant key information.

Extensions

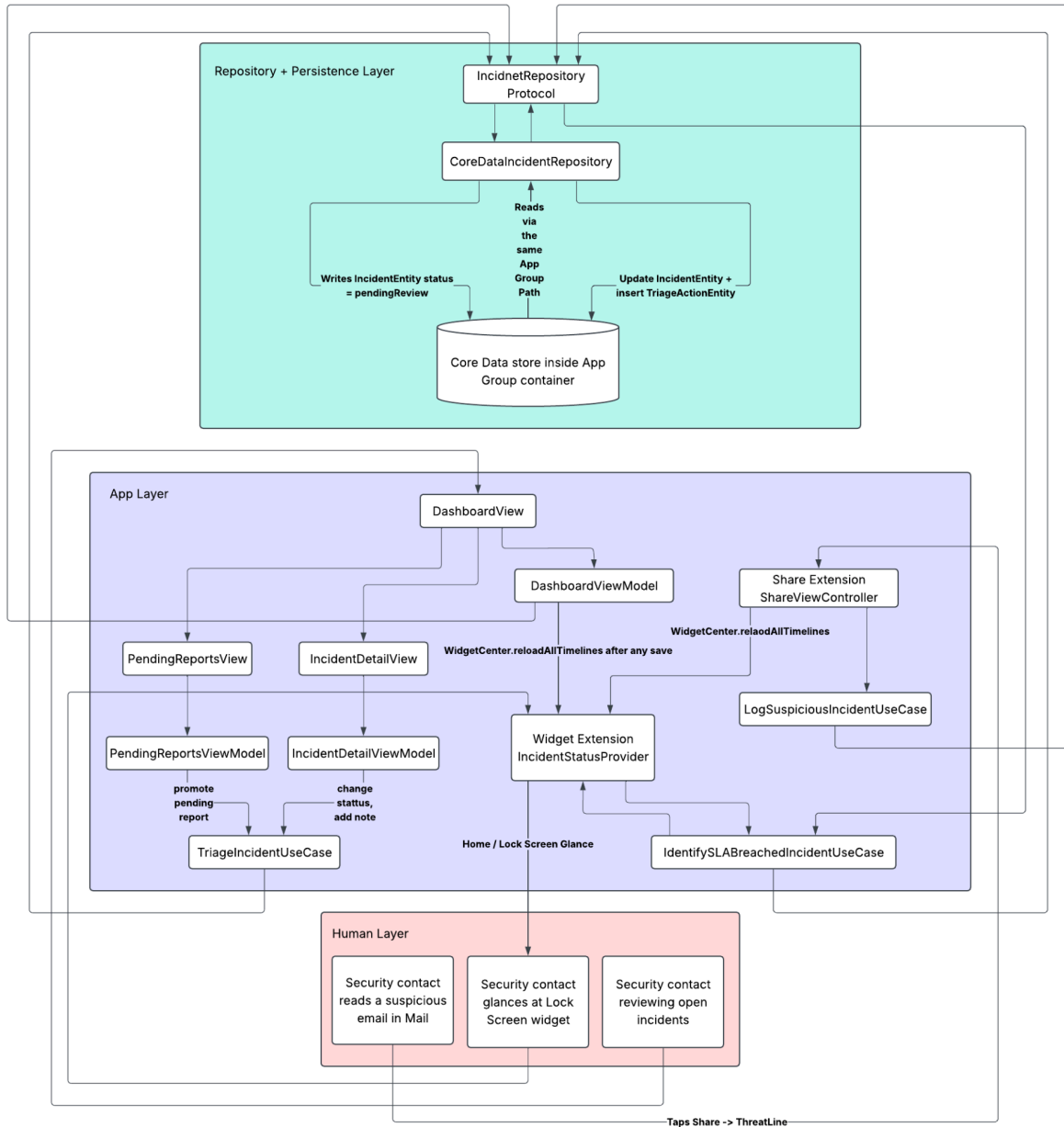
The share extension allows the security point of contact to share suspicious content straight from Mail/ Messages/ Safari into ThreatLine at the moment they notice it, rather than losing the context through constantly switching between apps. The widget extension shows the open-incident count and SLA-breach status at a glance, which is useful when in meetings or situations in which opening the full app isn't practical.

Database

Core Data was chosen over CloudKit because of the domain constraint of sensitive incident content. This information belongs to a stakeholder and it is preferable to not sync this to a cloud service by default, making this design choice an active and defensive security choice. Core data gives fast, offline first data and private storage only shared as far as the device's own App Group. This goes exactly as far as is needed for the app and stakeholder, making it the ideal choice.

Section 3 - Architecture Diagram

Diagram



Primary Use Case Data Flow

1. **User Action:** Security contact taps “Share -> ThreatLine” on a suspicious email in Mail.
2. **System Response:** ‘ShareViewController’ reads the shared text, constructs a call to ‘LogSuspiciousIncidentUseCase’.execute(...)’ with ‘intakeStatus: ,pendingReview’
3. **Use case:** Validates the content isn’t empty, checks for duplicate against a currently-open incident via ‘IncidentRepository.fetchOpenIncidents()’, then calls ‘repository.save(_:)’.
4. **Repository -> persistence:** ‘CoreDataIncidentRepository’ maps the domain struct to an ‘IncidentEntity’, writes it via ‘NSManagedObjectContext.perform’, and saves into the App Group shared container.
5. **System Response:** The extension calls
6. **User Action:** The widget’s ‘incidentStatusProvider.getTimeline’ reads ‘IdentifySLABreachedIncidentUseCase’, and renders the updated open-incident count.
7. **User Action:** Opens the dashboard, taps into ‘Pending Shared Reports’, assigns a real severity/ category, and ‘TriageIncidentUseCase’ promotes the incident to ‘.open’ writing both the updated ‘IncidentEntity’ and a new ‘TriageActionEntity’ in one Core Data transaction.

Note: The actual project has the functionality for extensions commented out because of the inability to use them due to not being able to create an Apple account and use it for code signing. This means this use case data flow could not be tested in actual development, but I chose to highlight it here to showcase my understanding of the use case data flow and how it would work.

Section 4- Reflective Report

Problem & justification:

I chose this problem after analysing my last assignment project. Following on from Vulnerability Dictionary I wanted to remain within the domain of cybersecurity, but approach a different angle. This is where the idea for ThreatLine arose, changing the focus from a professional environment focussed on addressing the needs of a dedicated cybersecurity team to a small organisational environment that can't rely on dedicated SOC (Security Operations Centre) or IT security teams and instead relies on security points of contact. After researching into some of the latest cyber reports I found evidence that this was a real problem and worth addressing, Paloalto Networks 2026 Unit 42 Global Incident Report detailing the following key quote that stuck with me: "Many organizations fail to leverage the telemetry needed to observe early-stage attacker behavior. Critical indicators of initial access and early attacker activity often go unnoticed because the SOC has not operationalized signals across endpoint, network, cloud and SaaS layers." My understanding of this problem deepened as I constructed the solution, showcasing how early-stage observations are missed due to the fallibility of tools, as security points of contact don't always have the right tools to detail the key information that needs to be recorded. This reinforced my belief in this app's potential to resolve the problem as it provides an easy method to record incidents.

Extension design decisions:

The share extension plays a key role in the app's usage as it allows the exact moment the security contact reads something suspicious in another app to be recorded. Without it capturing would rely upon switching out of the app into the security app, remembering details and typing it up, which creates friction in the ease of use of recording a potential incident for the security point of contact.

The widget extension provides the security point of contact with a user-friendly interface that they can view at a glance for situations in which they need a subtle or quick glance at recorded incidents. If removed the security point of contact may notice, but it would not be detrimental to the app's usability as it acts as a layer of convenience.

Database design decisions:

As noted previously in the design justification section the primary reasoning for choosing Core Data over Cloud Kit was prioritising security. Core Data provides offline data storage, minimising the access to sensitive data noted in the incident reports. The most significant schema decision was splitting 'IncidentEntity' and 'TriageActionEntity' into two related entities as opposed to a singular flat table with a 'status log' string field. This was implemented because a flat text log couldn't be queried or attribute individual actions to a person/ time reliably, making the Incident Detail screen's timeline hard to render cleanly.

Architecture under pressure:

When constructing the project a moment in which a technical requirement conflicted with user experience arose surrounding 'LogSuspiciousIncidentUseCase'. This use case needed a way to distinguish between incidents logged manually with full detail and incidents captured through the Share Extension with raw text. The cleanest technical solution was to have every incident start as 'open' and leave the severity category at a default deemed sensible. This would have been simpler but would have left un-triaged incidents with guessed severity levels sitting in the same list as genuinely reviewed incidents. This defeats the purpose of the system as it would no longer reflect real urgency. This was resolved by introducing 'IncidentIntakeStatus' as a separate restricted type and a distinct 'Pending Shared Reports' screen. These introduced an extra screen and use case parameter, but maintained the honesty of the incident's honesty.

AI tools:

AI coding assistants were used to enhance and improve the quality of code, while also providing useful feedback on how to approach and overcome obstacles. In nearly all cases this helped enhance and expedite the code writing process, leading to cleaner output with improved functionality. In regards to how the output was evaluated, it was checked against the business rules to ensure it maintained the structure of the app, and was tested to ensure it compiled and worked in the simulator. A defining example was in the writing of code for the extensions. Due to complications involving the inability to log into an Apple account the extensions could not be utilised as a code signer could not be assigned. AI coding tools were utilised to attempt to overcome this obstacle, leading to the construction of entitlement files which theoretically would have bypassed the need for an Apple account. Unfortunately, this approach failed so AI was utilised as a consultant for the code of the extensions instead, providing feedback and structure on what should theoretically produce working output.